Kantenkonsistenz und AC-3

Carsten Gips (HSBI)

TL;DR
Bei der Backtracking-Suche werden schrittweise Variablen belegt. Dabei kann eine Belegung eine Lösung im weiteren Verlauf der Suche unmöglich machen, so dass (viel) Backtracking notwendig wird.
Beim **Forward Checking** entfernt man nach der Belegung einer Variablen in allen Nachbarvariablen die durch die aktuelle Belegung inkonsistent gewordenen Werte. Wenn dabei ein Wertebereich leer wird, führt die aktuelle Belegung nicht zu einer Lösung und kann sofort zurückgenommen werden. Allerdings findet man mit Forward Checking nicht alle Inkonsistenzen.
Bei der **Kantenkonsistenz** prüft man, ob zu jedem Wert aus dem Wertebereich einer Variablen in den Nachbarvariablen mindestens ein passender (konsistenter) Wert existiert. Dabei werden die Constraints nacheinander betrachtet (nicht gleichzeitig). Wenn dies nicht der Fall ist, wird der Wert aus dem Wertebereich der betrachteten Variablen entfernt. Der AC-3-Algorithmus erzeugt schrittweise Kantenkonsistenz für ein CSP.
Man kann den AC-3 als Vorverarbeitung nutzen und die Wertemengen *vor* der BT-Suche reduzieren. Eventuell findet man dabei bereits eine Lösung oder kann eine Lösung ausschließen. Man kann den AC-3 auch als Inferenzschritt in die BT-Suche einbetten ("MAC").
Videos

- VL CSP, AC-3

---

1 Problem bei BT-Suche

Zuweisung eines Wertes an Variable X:

- Passt zu aktueller Belegung
- Berücksichtigt aber nicht **restliche** Constraints
  => macht weitere Suche u.U. unmöglich/schwerer

Lösung: Nach Zuweisung alle *nicht zugewiesenen Nachbarvariablen* prüfen

2 INFERENCE: Vorab-Prüfung (Forward Checking)

```
def BT_Search(assignment, csp):
    if complete(assignment): return assignment

    var = VARIABLES(csp, assignment)

    for value in VALUES(csp, var):
        if consistent(value, var, assignment, csp):
            assignment += {var = value}

            if INFERENCE(csp, assignment, var) != failure:
                result = BT_Search(assignment, csp)
                if result != failure: return result

            assignment -= {var = value}

    return failure
```

Quelle: Eigener Code basierend auf einer Idee nach (Russell und Norvig 2020, S. 176, Fig. 5.5)

Inference: Frühzeitiges Erkennen von Fehlschlägen! (vgl. (Russell und Norvig 2020, 178))

Nach Zuweisung eines Wertes an Variable X:

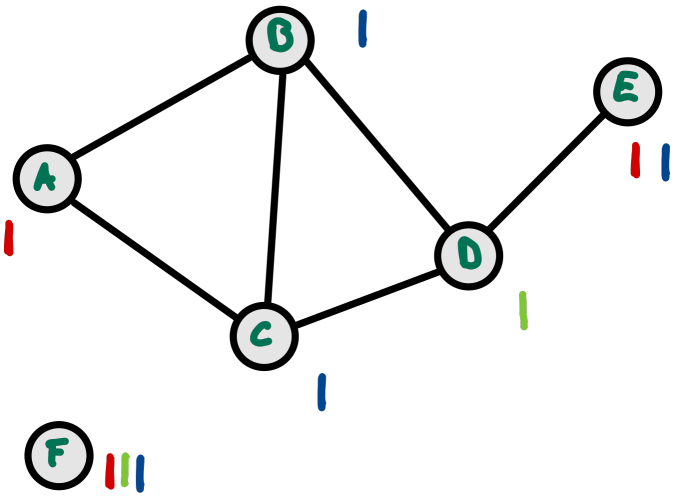
- Betrachte alle nicht zugewiesenen Variablen Y:
  - Falls Constraints zw. X und Y, dann …
  - … entferne alle inkonsistenten Werte aus dem Wertebereich von Y.

Beispiel:

1. Sei A auf rot gesetzt => entferne rot in B und C
2. Sei D auf grün gesetzt => entferne grün in B und C und E

Problem: Für B und C bleibt nur noch blau; sind aber benachbart!

3 Forward Checking findet nicht alle Inkonsistenzen!



- Nach $\{A = red, D = green\}$ bleibt für B und C nur noch blue
- B und C sind aber benachbart

4 Übergang von Forward Checking zu Kantenkonsistenz

- Forward Checking erzeugt Konsistenz für alle Constraints der **gerade betrachteten (belegten) Variablen**.

- Idee: Ausdehnen auf alle Kanten … => Einschränken der Wertemengen

5 Definition Kantenkonsistenz (Arc Consistency)

> Eine Kante von X nach Y ist "konsistent", wenn für jeden Wert $x \in D_X$ und für alle Constraints zwischen X und Y jeweils ein Wert $y \in D_Y$ existiert, so dass der betrachtete Constraint durch (x, y) erfüllt ist.

Ein CSP ist kanten-konsistent, wenn für alle Kanten des CSP Konsistenz herrscht.

6 Beispiel Kantenkonsistenz

$V = \{a, b, c, d, e\}$

$\mathrm{C} = \{((a, b), \neq), ((b, c), \neq), ((a, c), \neq), ((c, d), =), ((b, e), <)\}$

$D_a = D_b = D_c = \{1, 2, 3\}, D_d = \{1, 2\}, D_e = \{1, 2, 3\}$

Einschränkung der Ausgangswertemengen (kanten-konsistent)

$D_a = \{1, 2, 3\}, D_b = \{1, 2\}, D_c = \{1, 2\}, D_d = \{1, 2\}, D_e = \{2, 3\}$

=> Kantenkonsistenz ist nur **lokale** Konsistenz!

Anmerkung: $((a, b), \neq)$ ist Kurzform für $((a, b), \{(x, y) \in D_a \times D_b | x \neq y\})$

7 AC-3 Algorithmus: Herstellen von Kantenkonsistenz

```
def AC3(csp):
    queue = Queue(csp.arcs)
    while not queue.isEmpty():
        (x,y) = queue.dequeue()
        if ARC_Reduce(csp,x,y):
            if D_x.isEmpty(): return false
            for z in x.neighbors(): queue.enqueue(z,x)
    return true

def ARC_Reduce(csp, x, y):
    change = false
    for v in D_x:
        if not (any w in D_y and csp.C_xy(v,w)):
            D_x.remove(v);  change = true
    return change
```

Quelle: Eigener Code basierend auf einer Idee nach (Russell und Norvig 2020, p. 171, fig. 5.3)

Anmerkung: Die Queue in AC-3 ist wie eine (mathematische) Menge zu betrachten: Jedes Element kann nur genau einmal in einer Menge enthalten sein. D.h. wenn man bei `queue.enqueue(z,x)` die Rückkanten von den Nachbarn in die Queue aufnimmt, sorgt die Queue eigenständig dafür, dass es keine doppelten Vorkommen einer Kante in der Queue gibt. (Falls die verwendete Queue in einer Programmiersprache das nicht unterstützt, müsste man bei `queue.enqueue(z,x)` stets abfragen, ob die Kante `(z,x)` bereits in der Queue ist und diese dann nicht erneut hinzufügen.) AC-3 hat eine Laufzeit von $O(d^3n^2)$ (n Knoten, maximal d Elemente pro Domäne). Leider findet auch AC-3 nicht alle Inkonsistenzen … (NP-hartes Problem).

Hinweis: In gewisser Weise kann man Forward Checking als ersten Schritt bei der Herstellung von Kantenkonsistenz interpretieren.

8 Einsatz des AC-3 Algorithmus

1. Vorverarbeitung: Reduktion der Wertemengen *vor* BT-Suche
   - Nach AC-3 evtl. bereits Lösung gefunden (oder ausgeschlossen)

2. Propagation: Einbetten von AC-3 als Inferenzschritt in BT-Suche
   (**MAC** – Maintaining Arc Consistency)
   - Nach jeder Zuweisung an X_i Aufruf von AC-3-Variante:
     - Initial nur Kanten von X_i zu allen noch nicht zugewiesenen Nachbarvariablen
   - Anschließend rekursiver Aufruf von BT-Suche

9 Wrap-Up

- Anwendung von Forward Checking und …
- … die Erweiterung auf alle Kanten: AC-3, Kantenkonsistenz

10 Zum Nachlesen

- Russell und Norvig (2020): CSP, AC-3: Abschnitt 5.2
- Kumar (1992)
- Barták (2001)

Lernziele

- k2: Forward Checking (FC)
- k2: Erweiterung von FC auf alle Kanten: Kantenkonsistenz
- k2: Kantenkonsistenz bedeutet nicht globale Konsistenz
- k3: AC-3 Algorithmus

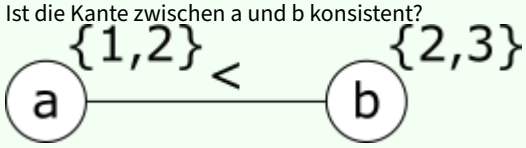
Quizzes

- Selbsttest CSP, AC-3 (ILIAS)

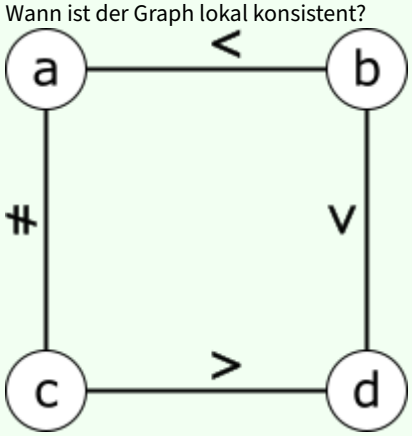
Challenges

Fingerübungen

Ist die Kante zwischen a und b konsistent?

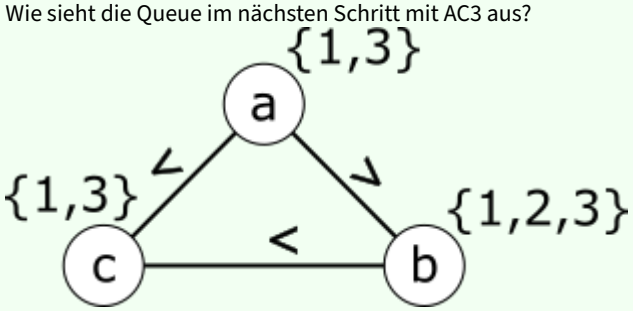


Wann ist der Graph lokal konsistent?



- a {1,2}; b {2,3}; c {1,2,3}; d {1,2,3}
- a {1,2}; b {2,3}; c {3}; d {1,2}
- a {1,3}; b {2,3}; c {1,3}; d {1,2,3}
- a {1,2}; b {2,3}; c {1,3}; d {1,2,3}

Wie sieht die Queue im nächsten Schritt mit AC3 aus?



Aktuelle Queue: [ab, ac, ba, bc, ca, cb]

- [bc, ba, ca, cb, ab, ac]
- [ab, ac, ba, bc, ca, cb]
- [ac, ba, bc, ca, cb]
- [ac, ba, bc, ca, cb, ba]

Quellen

Barták, R. 2001. „Theory and Practice of Constraint Propagation“. In *Proceedings of the 3rd Workshop on Constraint Programming for Decision and Control*. Bd. 50.

Kumar, V. 1992. „Algorithms for Constraint Satisfaction Problems: A Survey“. *AI Magazine* 13 (1): 32–44.

Russell, S., und P. Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4th Edition. Pearson. http://aima.cs.berkeley.edu.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- Eigener Code basierend auf einer Idee nach (Russell und Norvig 2020, p. 171, fig. 5.3)

> **Last modified:** 4fa1636 (lecture: use '?raw=true' for images via link to github repo, 2025-07-24)